

ShipShape

Auditing & improving a production TypeScript codebase from the U.S. Department of the Treasury.

// TARGET	// DURATION	// CATEGORIES	// GATE
Treasury/ship	7 days · Solo	7 — all must improve	Austin admission

WHAT THIS WEEK ASKS OF YOU

> Inherit a system you didn't build.
Understand it deeply. Measure its health.
Diagnose its weaknesses. **Make it better
with proof.**

The defining skill that separates junior from senior

WHAT WE'RE WORKING ON

Ship

A project management tool built by the U.S. Department of the Treasury — docs, issues, sprints, and real-time collaboration in one app.

- github.com/US-Department-of-the-Treasury/ship

stack.json

```

// monorepo · pnpm workspaces

frontend → React · Vite · Tailwind
editor → TipTap + Yjs
backend → Express · Node
database → PostgreSQL
realtime → WebSocket + Yjs CRDTs
testing → Playwright E2E (73+)
infra → Docker · Terraform

// architectural decision
model → "everything is a document"

```

THE CORE PRINCIPLE

read > write

You will spend more time understanding existing patterns than creating new ones. This week the audit is the deliverable. Reading code is not preparation for the work — reading code is the work.

HOW THE WEEK SPLITS

Two phases. One bar.

PHASE 01 · 36 HOURS

The Audit

// diagnosis

A written report with baseline measurements for all seven categories. Tools, methodology, raw data, ranked findings. You do not fix anything during the audit.

HARD GATE Incomplete = automatic fail

PHASE 02 · 4.5 DAYS

Implementation

// treatment

Measurable improvement in every category — not pick-three. Before/after benchmarks, identical conditions, tests still passing, root cause documented.

SCORED 40% of grade · measurable improvement

THE SPINE OF THE WEEK

The seven categories

01

Type Safety

any · as · ! · @ts-ignore · strict mode

02

Bundle Size

treemap · chunks · code splitting · dead deps

03

API Response Time

P50 · P95 · P99 · concurrent load

04

Database Queries

N+1 · indexes · EXPLAIN ANALYZE

05

Test Coverage

73 Playwright tests · gaps · flakes

06

Runtime Errors

boundaries · network failure · malformed input

07

Accessibility

Lighthouse · axe · WCAG 2.1 AA · keyboard

// TOTAL

7

measurable improvement targets

WHAT PASSING LOOKS LIKE

The bar, per category

#	CATEGORY	TARGET	GATE
01	Type Safety	Eliminate 25% of type-safety violations · proper narrowing required	PASS
02	Bundle Size	15% total reduction · or 20% off initial via code split	PASS
03	API Response Time	20% P95 reduction on ≥2 endpoints · identical conditions	PASS
04	Database Queries	20% fewer queries on 1 flow · or 50% on slowest query	PASS
05	Test Coverage	+3 meaningful tests on untested paths · or fix 3 flakes w/ RCA	PASS
06	Runtime Errors	+3 error-handling fixes · ≥1 real data-loss scenario	PASS
07	Accessibility	+10 Lighthouse on worst page · or all Critical/Serious on top 3	PASS

HOW SENIOR ENGINEERS WORK

Rules of engagement

01 Before/after is mandatory

// reproducible benchmark, identical conditions, every category

02 Tests must still pass

// break a test → fix it with justification, or revert the change

03 Document your reasoning

// what changed · why old was bad · why new is better · tradeoffs

04 No cosmetic changes

// renames, reformats, comments don't count unless they move a metric

05 Commit discipline matters

// labeled branches · descriptive messages · they read your git history

06 Depth over breadth

// targeted, well-documented fixes beat a scattered attempt at everything

OUR DATES

Seven days.

Three deadlines.



// TODAY

Kickoff

+ 4h orientation

Clone, read, map. No measuring yet.



// WEDNESDAY

MVP

12:00 PM CT

Audit report complete
+ first round of fixes landed



// FRIDAY

Early

12:00 PM CT

Optional early submission
for feedback pass



// SUNDAY

Final

10:59 PM CT

All deliverables shipped.
Gate for Austin admission.



Phase 01 — First Contact

// 01 Repository Overview

- ›

Clone, run, doc the gaps

› Every install step that wasn't in the README

› What broke. What you had to guess.
- ›

Read every file in docs/

› Summarize each decision in your own words

› Note anything that feels load-bearing
- ›

Read shared/ end-to-end

› This is the contract between front and back

› Diagram how web/ ↔ api/ ↔ shared/ connect

// 02 Data Model

- ›

Map the schema

› From migrations and seed files

› Tables + relationships, drawn out
- ›

Everything is a document

› One documents table for docs, issues, projects, sprints

› Understand the document_type discriminator
- ›

Relationships

› Linking · parent-child · project membership

› How they're queried at runtime

// 03 Request Flow

- ›

Trace one user action E2E

› Pick "create issue" – follow it through

› React component → API route → DB query → back
- ›

Middleware chain

› What runs before every API request?

› Order, side effects, where it can short-circuit
- ›

Authentication

› How does auth actually work?

› What happens to an unauthenticated request?

Phase 02 — Deep Dive · Phase 03 — Synthesis

// 04 Real-time Collab

- › WebSocket connection
- › How it gets established + authed
- › Presence + cursor tracking messages

- › Yjs CRDT sync
- › How document state syncs between users
- › Two users edit the same field — what happens?

- › Server persistence
- › Where + when Yjs state is saved
- › What's the source of truth on reconnect?

// 05 TypeScript Patterns

- › Config + version
- › TS version · tsconfig.json · strict mode?
- › Per-package overrides

- › Find examples of...
- › Generics · discriminated unions
- › Utility types · type guards

- › Unknown patterns
- › See something you don't recognize?
- › Research it. That's a discovery candidate.

// 06 · 07 Tests + Infra

- › Playwright suite
- › Structure · fixtures · how the test DB resets
- › Run it. Pass count + runtime are your baseline.

- › Build & deploy
- › Dockerfile · docker-compose services
- › Skim Terraform: what cloud does it expect?

- › CI/CD
- › Pipeline (if configured) · gates · cadence

A DELIVERABLE ON THE RUBRIC

Find three things you didn't know.

01

// WHAT

A TypeScript feature, library, architectural pattern, or engineering practice that is new to you.

// WHERE

file path + line range

02

// WHY IT MATTERS

What it actually does in the system and why the team made that choice.

// THEN

how you'd apply it next time

03

// THE POINT

This is the thing you'll remember **in your next job** — not the metric you moved by 22%.

// KEEP

a running notes file all week

WHAT GOES IN THE BOX

Submission checklist



Forked GitHub repo

// improvements on labeled branches · setup guide in README



Improvement docs

// before · root cause · fix · after · proof – per category



Demo video

// 3-5 min · walk through findings + before/after



Deployed application

// improved fork · running · publicly accessible



Audit report

// baselines for all 7 · methodology · tools · raw data



Discovery write-up

// 3 things · codebase refs · reflection



AI cost analysis

// spend + reflection on AI for codebase comprehension



Social post

// X or LinkedIn · key findings · tag @GauntletAI

WHAT GETS MEASURED

How it's graded

Measurable improvement	40%
Technical depth	25%
TypeScript quality	15%
Documentation quality	10%
Commit discipline	10%

~1/2 the grade is "**can you prove it.**"
Measurement and writing aren't the boring parts of the week — **they are the week.**

 READY · SET

Depth over breadth. Proof over promises.

● ● ● ~/shipshape

```
$ git clone github.com/US-Department-of-the-Treasury/ship
```

```
$ cd ship && pnpm install
```

```
$ git checkout -b shipshape/audit
```

```
↳ Switched to branch 'shipshape/audit'
```

```
// see you Wednesday, noon CT.
```

```
$ █
```